AN EVIDENCE-ADAPTIVE, OPTICAL CHARACTER RECOGNITION (OCR)-RESILIENT HYBRID RETRIEVAL FRAMEWORK FOR DOCUMENT QUESTION ANSWERING

A Project Report

Submitted by

MANTHIRA MOORTHY P (Reg. no. 9517202303083)

VIJAY S K (Reg. no. 9517202303175)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

MEPCO SCHLENK ENGINEERING COLLEGE, SIVAKASI

(An Autonomous Institution affiliated to Anna University Chennai)

Register No: 9517202303083, 9517202303175

CERTIFICATE

This is to certify that it is the Bonafide work done by Mr. MANTHIRA MOORTHY P (9517202303083), Mr. VIJAY S K (9517202303175) for the project work entitled An Evidence-Adaptive, OCR-Resilient Hybrid Retrieval Framework for Document Question Answering at the Department of Electronics and Communication Engineering, Mepco Schlenk Engineering College, Sivakasi.

Note on this certificate page: this project's course code, submission date, guide, Principal, and Head of Department were not reused from the old mini-project report (which concerned a different project, Mini Project IV, submitted May 2026, under a different guide) per explicit instruction. The guide's name below is taken directly from this project's own source documentation (`PROJECT_SCOPE.md`: "Mentor: Dr. V. Karthikeyan, AP(Sl.G.)"). All other institutional fields are placeholders for the responsible authority to complete and sign.

..... | .....

Faculty In-Charge (Dr. V. Karthikeyan) | Head of the Department [NAME TO BE FILLED]

Submitted for the Practical Examination held at Mepco Schlenk Engineering College, Sivakasi on [DATE TO BE FILLED]

..... | .....

Internal Examiner - I | Internal Examiner - II

ACKNOWLEDGEMENT

The authors express their sincere thanks to the Department of Electronics and Communication Engineering, Mepco Schlenk Engineering College, Sivakasi, for extending the facilities required to complete this project. The authors record their deepest gratitude to their project guide, Dr. V. Karthikeyan, Assistant Professor (Sl. Grade), for his valuable guidance and consistent encouragement throughout the course of this work. **[The Principal's and the Head of the Department's names are intentionally left as placeholders here — they were not confirmed to this workflow and should be inserted and verified before submission, per the explicit instruction not to reuse the old report's institutional names for a different project and a different year.]**

ABSTRACT

Retrieval-Augmented Generation (RAG) systems retrieve a fixed number of passages from a document collection and generate an answer unconditionally, regardless of whether the retrieved evidence actually supports a correct answer. This weakness is inherited directly from QuIM-RAG (Saha et al., IEEE Access, 2024), the question-indexed retrieval baseline this project extends, and is made worse for scanned documents, where Optical Character Recognition (OCR) noise, broken layouts, and ambiguous retrieval make silent wrong answers more likely. This project implements and evaluates an OCR-resilient, hybrid-retrieval document question-answering system that adds Evidence-Adaptive Retrieval and Verification (EAR): a calibrated decision layer combining four signals already available at retrieval time — dense/lexical retrieval agreement, propagated OCR confidence, cross-passage numeric contradiction, and query complexity — into a logistic sufficiency score, used to decide whether the system should answer, expand its retrieval, inspect structure, or abstain. On the OHR-Bench benchmark, restricting answers to EAR's confident decisions raised top-1 retrieval accuracy from 71.2% to 78.3% (a 7.1-percentage-point improvement, 95% confidence interval [+2.5, +12.0], n = 252), and this result was independently reproduced on a second, disjoint 70-query sample (84.3% to 96.0%). On the project's own generated answers, EAR-gating also raised RAGAS Faithfulness from 0.514 to 0.619, at a computational cost roughly 1% of an equivalent large-language-model-based verification step. The system additionally closes two gaps present in the original QuIM-RAG design — a real BM25 lexical retrieval path and Residual Vector Quantization for index compression — both evaluated honestly, including configurations where they did not improve on the baseline. The complete system was implemented as an interactive Streamlit application with OCR ingestion, hybrid retrieval, EAR-gated answering, and a full statistical evaluation harness, verified by an automated test suite of 67 passing tests. Limitations — including an OCR-confidence signal not yet validated on genuinely noisy (non-synthetic) OCR data, and a substitute-data reproduction that did not replicate QuIM-RAG's own reported advantage — are reported explicitly rather than concealed.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	ACKNOWLEDGEMENT	iv
	LIST OF TABLES	v
	LIST OF FIGURES	vi
	LIST OF ABBREVIATIONS	vii
1	**INTRODUCTION**	1
	1.1 Project Background	1
	1.2 Objectives of the Project	-
	1.3 Organization of the Report	-
2	**LITERATURE SURVEY**	-
	2.1 Review of Related Work	-
3	**DESIGN METHODOLOGY**	-
	3.1 Design Parameters of the Proposed System	-
	3.2 Software Module Description	-
	3.3 Formulae Used in the Design	-
	3.4 System Design and Implementation Steps	-
	3.5 Detailed System Workflow	-
	3.6 System Architecture	-
	3.7 User Interface and Functional Modules	-
	3.8 Data Storage, Indexing, and Retrieval Design	-
4	**RESULTS AND DISCUSSION**	-
	4.1 System Operation Verification	-
	4.2 Document Ingestion and OCR Evaluation	-
	4.3 Data Processing and Indexing	-
	4.4 Retrieval Configuration Comparison	-
	4.5 EAR Decision Calibration and Selective Prediction	-

CHAPTER NO.	TITLE	PAGE NO.
	4.6 Ablation Study and Signal Analysis	-
	4.7 RVQ Compression and Retrieval Trade-Off	-
	4.8 Answer Generation and RAG Evaluation	-
	4.9 Performance Visualization and Analysis	-
	4.10 Automated Testing and Practical Usability	-
	4.11 Comparison with the Existing QuIM-RAG System	-
	4.12 Limitations and Research-Gap Validation	-
5	**CONCLUSION AND FUTURE SCOPE**	-
	5.1 Conclusion	-
	5.2 Major Contributions	-
	5.3 Limitations	-
	5.4 Future Scope	-
	REFERENCES	-

(Page numbers beyond this point are left as "-" rather than guessed: an accurate page number can only be known once the document is paginated by the reading application, and this report's own validation record states this explicitly rather than presenting invented numbers as real pagination.)

LIST OF TABLES

TABLE NO.	TABLE TITLE	PAGE NO.
3.1	Hardware and Software Requirements	-
4.1	Retrieval Configuration Comparison	-
4.2	EAR Calibrated Weights	-
4.3	Leave-One-Signal-Out Ablation	-
4.4	EAR Decision Groups and Accuracy	-
4.5	RVQ Compression/Recall Trade-Off	-
4.6	RAGAS Metrics, Baseline vs. EAR-Gated	-
4.7	pytest Test Module Summary	-
4.8	Substitute-Data Reproduction, Recall@5	-
4.9	Comparative Analysis with the Existing QuIM-RAG System	-

LIST OF FIGURES

FIGURES NO.	FIGURES TITLE	PAGE NO.
3.1	System Architecture: Offline Indexing and Online Query Processing	-
3.2	EAR Signal Computation and Decision Logic	-
4.1	OCR Confidence vs. Measured Character Error Rate	-
4.2	Retrieval Configuration Comparison Graph	-
4.3	EAR Selective-Prediction Accuracy by Decision Group	-
4.4	RVQ Compression vs. Recall@5	-
4.5	RAGAS Faithfulness, Baseline vs. EAR-Gated	-
4.6	Substitute-Data Reproduction Comparison	-

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
RAG	Retrieval-Augmented Generation
OCR	Optical Character Recognition
QA	Question Answering
EAR	Evidence-Adaptive Retrieval and Verification
RVQ	Residual Vector Quantization
BM25	Best Matching 25
RRF	Reciprocal Rank Fusion
CER	Character Error Rate
WER	Word Error Rate
RAGAS	Retrieval-Augmented Generation Assessment
LLM	Large Language Model
CI	Confidence Interval
nDCG	Normalized Discounted Cumulative Gain
MRR	Mean Reciprocal Rank
MAP	Mean Average Precision

CHAPTER 1

INTRODUCTION

1.1 PROJECT BACKGROUND

With the increasing volume of institutional, legal, financial, and technical documents maintained in scanned and digital form, the need for systems that can automatically answer natural-language questions over large document collections has grown significantly. Retrieval-Augmented Generation (RAG) has emerged as the dominant approach for this task, combining an information-retrieval component with a large language model so that answers are grounded in retrieved evidence rather than relying solely on a model's internal, potentially outdated knowledge. This project is motivated by a specific weakness shared by most RAG systems, including the question-indexed retrieval approach (QuIM-RAG) that it directly extends: once a fixed number of passages is retrieved, the system answers unconditionally, with no mechanism to recognise that the retrieved evidence might be wrong, incomplete, or internally contradictory.

1.1.1 INTRODUCTION TO DOCUMENT QUESTION ANSWERING:

Document Question Answering (DQA) is the task of producing a correct, evidence-grounded answer to a natural-language question posed against a large collection of documents. Unlike closed-book question answering, where a model answers purely from parameters learned during training, DQA systems must locate the relevant passage or passages within the document collection before an answer can be generated. This becomes considerably harder when the document collection is large, heterogeneous in format (reports, manuals, legal filings, scanned forms), or contains recognition noise introduced by scanning and Optical Character Recognition.

1.1.2 INTRODUCTION TO RETRIEVAL-AUGMENTED GENERATION:

Retrieval-Augmented Generation (RAG) addresses closed-book question answering's knowledge-cutoff and hallucination problems by retrieving relevant passages from an external corpus and supplying them to a large language model as context before generation. A typical RAG pipeline embeds document chunks into a vector space, retrieves the top-k chunks most similar to a query embedding, and passes those chunks to the generator along with the original question. QuIM-RAG (Saha, Saha, and Malik, IEEE Access, 2024) refines this idea by generating candidate questions for each document chunk and indexing those questions rather than the chunks themselves, so that

a user's query is matched against semantically similar questions instead of raw passages — a design intended to reduce the information dilution caused by fixed-size chunking. However, like most RAG systems, QuIM-RAG always generates an answer once retrieval has returned its top-k results, without any mechanism to detect when the retrieved evidence is actually insufficient or unreliable.

1.1.3 INTRODUCTION TO OCR AND OCR-RELATED CHALLENGES:

Optical Character Recognition (OCR) converts scanned page images into machine-readable text and is a necessary preprocessing step whenever a document collection includes non-digital-native sources such as scanned forms, old reports, or photographed pages. OCR is imperfect: it introduces character substitutions, missing words, broken table structure, and layout distortion, particularly on low-quality scans or unusual fonts. These errors propagate forward into every downstream stage of a RAG pipeline — a mis-recognised word can cause a relevant passage to be missed by lexical retrieval, and a corrupted number can cause an answer to be wrong even when the correct passage was retrieved. QuIM-RAG's original evaluation used cleanly scraped, natively digital website text and never evaluated its retrieval behaviour under OCR noise at all, leaving this a completely open question for the approach.

1.1.4 ADVANTAGES AND LIMITATIONS OF HYBRID RETRIEVAL:

Hybrid retrieval combines a dense (embedding-based) retriever, which captures semantic similarity, with a lexical (keyword-based) retriever such as BM25, which captures exact term and numeric matches that dense embeddings can miss. The two ranked lists are typically combined using Reciprocal Rank Fusion (RRF). The principal advantage of hybrid retrieval is that it is robust to a wider range of query types than either retriever alone — dense retrieval tends to fail on queries containing specific identifiers, codes, or numbers, while lexical retrieval tends to fail on queries that are semantically similar to a passage but use different wording. The main limitation, confirmed experimentally in this project (Chapter 4), is that hybrid fusion does not always outperform its individual components: on the benchmark used here, BM25 alone outperformed the fused hybrid configuration on one of the principal retrieval-quality metrics, a result that is reported honestly rather than adjusted to favour the hybrid design.

1.1.5 APPLICATIONS OF THE PROPOSED SYSTEM:

The proposed OCR-resilient, evidence-adaptive document question-answering system is applicable wherever natural-language access to a large, possibly scanned document collection is required and incorrect answers carry a real cost. Representative application domains include:

- Legal and regulatory document review, where a confidently wrong answer can have compliance consequences.
- Financial report analysis, where exact numeric accuracy matters more than semantic approximation.
- Technical manual and standard-operating-procedure lookup in engineering and manufacturing settings.
- Academic and institutional knowledge bases built from scanned archival material.
- Any document collection where selective abstention (declining to answer when evidence is weak) is preferable to a fluent but unsupported answer.

1.2 OBJECTIVES OF THE PROJECT

The objectives of this project are:

1. To extend the QuIM-RAG question-indexed retrieval design with OCR-aware document ingestion that captures real per-word OCR confidence, rather than discarding it after text extraction.
2. To add a real lexical (BM25) retrieval path fused with dense retrieval via Reciprocal Rank Fusion, recovering queries that hinge on exact identifiers or numeric values that dense embeddings alone can miss.
3. To design, implement, and evaluate Evidence-Adaptive Retrieval and Verification (EAR), a calibrated decision layer that decides whether a given query should be answered, have its retrieval expanded, have its evidence structure surfaced for inspection, or be abstained on, instead of the system always answering.
4. To evaluate the complete system honestly and reproducibly against a real, independent, publicly available benchmark (OHR-Bench), reporting both favourable and unfavourable findings.
5. To build a working, interactively explorable reference implementation (a Streamlit application) and verify it with an automated test suite.

1.3 ORGANIZATION OF THE REPORT

This report is organised into five chapters. Chapter 1 introduces document question answering, Retrieval-Augmented Generation, OCR-related challenges, and the objectives of the project. Chapter 2 reviews the literature relevant to QuIM-RAG, hybrid retrieval, OCR's effect on RAG, evidence-based selective prediction, and vector

quantization, concluding with the research gap this project addresses. Chapter 3 describes the design methodology, including design parameters, software modules, the mathematical formulation of each component, the system architecture, and the data-storage design. Chapter 4 presents the implementation results, experimental evaluation, and discussion, covering system verification, OCR evaluation, retrieval comparison, EAR calibration and validation, ablation analysis, RVQ trade-offs, RAGAS evaluation, automated testing, and a comparison with the existing QuIM-RAG system. Chapter 5 concludes the report, summarising the major contributions, limitations, and future scope.

CHAPTER 2

LITERATURE SURVEY

2.1 REVIEW OF RELATED WORK

The foundation for this project is QuIM-RAG, proposed by Saha, Saha, and Malik in IEEE Access (2024) [1]. QuIM-RAG generates candidate questions for each corpus chunk using GPT-3.5-turbo-instruct, builds an inverted index from a quantized question-embedding space to the source chunk, and retrieves by matching a user query against this question index rather than against chunk embeddings directly, reducing the information dilution caused by fixed-size chunking. Evaluated on two scraped university websites (731 total chunks, 22,609 generated questions) with a LLaMA3-8B-instruct generator and BAAI/bge-large-en-v1.5 embeddings, QuIM-RAG improved Faithfulness from 0.72 to 1.00 and BERTScore F1 from 0.36 to 0.67 over a traditional chunk-cosine RAG baseline on their custom question-and-answer dataset. However, the paper's corpus contained no OCR noise, used no lexical retrieval path, included no reranking stage, and always generated an answer regardless of retrieval confidence — three concrete limitations that motivate the present project.

The broader Retrieval-Augmented Generation literature establishes several mitigations for RAG's known weaknesses individually, though none in combination with each other or with QuIM-RAG's own design. Amaral Cejas, Guo, and Tang [2] show that embedding-model choice measurably changes retrieval accuracy and downstream answer faithfulness, motivating this project's choice to keep the embedding backend swappable (a MiniLM sentence embedding model with a TF-IDF fallback) rather than committing to a single fixed model. Dense retrieval, which compares query and passage embeddings by cosine similarity, is complemented in this project by a real lexical retrieval path using BM25, the probabilistic ranking function formalised by Robertson and Zaragoza [9]; the two ranked lists are combined using Reciprocal Rank Fusion, a rank-based fusion method that does not require score normalisation across retrievers. Elkiran and Rasheed's EvaRAG study [3] shows that indexing strategy and similarity-metric choice materially affect retrieval effectiveness and latency, supporting this project's practice of reporting latency alongside every accuracy number rather than accuracy in isolation. In a follow-up empirical evaluation, the same authors [4] find that retriever and reranker choice has the largest measured effect on final question-answering quality of any pipeline component tested, which directly motivated treating hybrid retrieval together with optional cross-encoder reranking as separately benchmarked

configurations in this project's own evaluation harness (Chapter 4), rather than as a single fixed pipeline.

OCR's effect on Retrieval-Augmented Generation is directly addressed by Zhang et al.'s OHR-Bench study [6], accepted at the IEEE/CVF International Conference on Computer Vision (ICCV) 2025, which shows that OCR noise measurably cascades into retrieval and generation degradation in RAG pipelines. OHR-Bench's released benchmark, comprising labelled question-answer pairs with page-level ground truth across real-world document domains, is the dataset this project's own experiments are evaluated against (Chapter 4), since QuIM-RAG's private NDSU corpus is not publicly available. Kiss, Nagy, and Szilágyi [5] propose Max–Min semantic (similarity-clustering) chunking and show that it outperforms fixed-size chunking on retrieval accuracy; the present project still uses paragraph- and sentence-boundary chunking with overlap, not a learned semantic-chunking algorithm, a gap that is tracked explicitly rather than silently closed. Sarthi et al.'s RAPTOR approach [7] motivates hierarchical, tree-organised retrieval over flat chunking; this project's question-tree component is a rule-based keyword grouping with templated question generation, not a trained recursive summarisation tree, a genuine scope difference.

The concept of evidence confidence and selective prediction — deciding whether a system's output is trustworthy enough to present, rather than always presenting an output — is well established in machine learning generally but is largely absent from the RAG literature reviewed for this project, including from QuIM-RAG itself. Evidence-Adaptive Retrieval and Verification (EAR), the principal contribution of this project, is designed specifically to close this gap: it combines four signals computed from retrieval that has already taken place — agreement between dense and lexical retrieval rankings, propagated OCR confidence, cross-passage numeric contradiction, and query complexity — into a calibrated logistic sufficiency score, used to decide whether the system should answer, expand retrieval, inspect structure, or abstain.

Vector quantization, used in this project to compress stored embeddings, is grounded in Jégou, Douze, and Schmid's Product Quantization method [8], of which this project's two-stage Residual Vector Quantization (RVQ) is a simplified instance, without the per-subspace splitting that production approximate-nearest-neighbour systems typically add. Finally, RAG evaluation methodology in this project follows the metric family used by QuIM-RAG itself — Faithfulness, Answer Relevancy, Context Precision, Context Recall, and Harmfulness, collectively known as RAGAS — implemented independently in this project and judged by a locally hosted language model rather than a paid cloud API.

Research gap addressed by this project. Taken together, the reviewed literature shows that embedding choice, indexing strategy, reranking, semantic chunking, and OCR noise each measurably affect Retrieval-Augmented Generation quality in isolation, but QuIM-RAG — the specific baseline this project extends — combines none of the mitigations the wider literature suggests, and no reviewed work, including QuIM-RAG, provides a mechanism for a RAG system to recognise and act on its own retrieval uncertainty. This project addresses that gap directly by adding OCR-aware ingestion, a real lexical retrieval path, and Evidence-Adaptive Retrieval and Verification to a hybrid-retrieval extension of QuIM-RAG's question-indexed design.

CHAPTER 3

DESIGN METHODOLOGY

3.1 DESIGN PARAMETERS OF THE PROPOSED SYSTEM

The design parameters of the proposed document question-answering architecture are as follows:

- Document quality and OCR processing
- Text chunking and embedding parameters
- Dense and lexical retrieval parameters
- EAR model and decision thresholds
- Training, calibration, and evaluation strategy
- Software and hardware environment

are essential for designing a reliable, OCR-resilient document question-answering system.

3.1.1 DOCUMENT QUALITY AND OCR PROCESSING:

Input documents may be natively digital PDFs or scanned images. Digital PDFs are text-extracted directly using PyMuPDF; pages without an extractable text layer fall back to Tesseract OCR, which also yields real per-word confidence scores that are retained per chunk rather than discarded after extraction. This per-word confidence later feeds directly into the EAR decision layer as one of its four signals.

3.1.2 TEXT CHUNKING AND EMBEDDING PARAMETERS:

Extracted text is split into paragraph- and sentence-aware chunks of 950 characters with a 100-character overlap, chosen to preserve sentence-level context across chunk boundaries. Each chunk is embedded using a MiniLM sentence embedding model, with a TF-IDF representation used as a fallback when embedding infrastructure is unavailable.

3.1.3 DENSE AND LEXICAL RETRIEVAL PARAMETERS:

Dense retrieval ranks chunks by cosine similarity between the query embedding and each chunk embedding. Lexical retrieval uses a real BM25 index (`rank_bm25.BM25Okapi`). The two ranked lists are fused using Reciprocal Rank Fusion with fusion constant k = 60, and an optional cross-encoder reranking stage may

be applied to the fused candidates before the top results are returned.

3.1.4 EAR MODEL AND DECISION THRESHOLDS:

The EAR sufficiency score is a logistic combination of four signals — agreement, OCR confidence, contradiction, and complexity — with weights fit by logistic regression on a calibration split that is never used for evaluation. Two thresholds govern the decision: `threshold_answer`, the lowest calibration probability reaching a precision of at least 0.8 for predicting top-1 correctness, and `threshold_abstain`, the 20th percentile of calibration probabilities. Both thresholds are fit from data, not hand-picked.

3.1.5 TRAINING, CALIBRATION, AND EVALUATION STRATEGY:

The OHR-Bench-derived dataset used for evaluation is split into a calibration set (used only to fit EAR's weights and thresholds) and a disjoint evaluation set (used for every reported headline result), in a 40:60 ratio. This separation ensures that EAR's reported improvement is measured on data it was never calibrated against. A second, entirely independent 70-question-pair sample (disjoint from both the calibration and evaluation sets) is used to validate the EAR result a second time (Chapter 4).

3.1.6 SOFTWARE AND HARDWARE ENVIRONMENT:

Table 3.1. Hardware and Software Requirements.

CATEGORY	REQUIREMENT
Development machine	Apple-silicon Mac (CPU/MPS used for all experiments; no GPU required)
Language / runtime	Python 3.11
Retrieval and machine-learning libraries	scikit-learn, rank_bm25, sentence-transformers, numpy, pandas
OCR	Tesseract OCR via pytesseract, with real per-word confidence
PDF extraction	PyMuPDF
Local language model	Ollama, running llama3.2:latest / llama3.2:1b (no API key, no cost)
Optional cloud language model	Anthropic API (user-supplied key; never written to disk or logged)
User-interface framework	Streamlit
Evaluation dataset	OHR-Bench (Zhang et al., ICCV 2025)
Automated testing	pytest

3.2 SOFTWARE MODULE DESCRIPTION

The implementation is organised as a Python package (`quimrag/`) consisting of the following modules:

- **Document extraction and retrieval engine (`core.py`):** performs document extraction including real OCR confidence capture, chunking, embedding, and the base dense/lexical retrieval engine.
- **Lexical retrieval module (`bm25.py`):** a wrapper around a real BM25 implementation, replacing an earlier substring-overlap lexical scorer.
- **Evidence-Adaptive Retrieval and Verification module (`adaptive.py`):** computes the four EAR signals, fits and applies the logistic sufficiency model, and implements the answer/expand/inspect/abstain decision logic — this project's principal original contribution.
- **Quantization module (`rvq.py`):** implements two-stage Residual Vector Quantization for compressing stored embeddings.
- **Orchestration module (`pipeline.py`):** chains query rewriting, multi-query retrieval fusion, answer generation, and a groundedness pre-check.
- **Query rewriting module (`query_rewrite.py`):** heuristic query-rewriting variants, with optional language-model-assisted rewriting.
- **Hierarchical question-tree module (`question_tree.py`):** a rule-based, keyword-driven question hierarchy attached to a document-section-chunk structure.
- **Evaluation module (`metrics.py`):** standard information-retrieval metrics, bootstrap confidence intervals, and paired significance tests.
- **Answer generation modules (`local_llm.py`, `llm_answer.py`):** local (Ollama) and optional cloud (Anthropic API) grounded answer generation, with a no-API extractive fallback.
- **RAGAS evaluation module (`ragas_eval.py`):** this project's own implementation of the RAGAS metric family, judged by a local language model.
- **Generation-quality module (`generation_eval.py`):** Exact Match, token-F1, semantic similarity, BERTScore, citation precision/recall, and numeric answer correctness.
- **Groundedness module (`hallucination_eval.py`):** a heuristic groundedness pre-check and human-in-the-loop validation workflow.

- **OCR evaluation module (`ocr_eval.py`):** formal Character Error Rate and Word Error Rate computation.
- **Diagnostics module (`diagnostics.py`):** exposes per-stage retrieval internals for inspection without altering retrieval behaviour.
- **Telemetry module (`telemetry.py`):** live central-processing-unit and memory statistics.
- **Baseline reproduction module (`quim_baseline.py`):** a faithful reproduction of QuIM-RAG's own question-matching retrieval mechanism, used for the comparison study in Chapter 4.

3.3 FORMULAE USED IN THE DESIGN

The operation of this system relies on retrieval-ranking theory, vector quantization, and statistical calibration. The key equations used in the system design are presented below.

1. Reciprocal Rank Fusion. Given n ranked candidate lists (Dense-only and BM25-only), the fused score for candidate d is:

$$S_{RRF}(d) = \sum_j \left[1 / (k + \text{rank}_j(d)) \right], k = 60 \quad ... (3.1)$$

Where: $\text{rank}_j(d)$ is the rank of candidate d in list j, and k is the fusion constant.

2. Residual Vector Quantization. Given an original embedding vector x and initial residual $r_0 = x$, at each stage m a code vector is selected from codebook C_m and the residual is updated:

$$c_m = \text{argmin}_{c \in C_m} \|r_{m-1} - c\|^2, \quad r_m = r_{m-1} - c_m \quad ... (3.2)$$

Where: the reconstructed vector is $\hat{x} = \sum(c_m)$ over all stages, and the quantization error is $E_{RVQ} = \|x - \hat{x}\|^2$.

3. Logistic Sufficiency Scoring (EAR). The four EAR signals combine into a sufficiency score:

$$p = \text{sigmoid}(w_0 + w_1 a + w_2 o + w_3 x + w_4 c) \quad ... (3.3)$$

Where: a = agreement (Jaccard overlap of Dense-only and BM25-only top-K chunk ids), o = mean OCR confidence of the retrieved evidence, x = fraction of evidence pairs that are topically related but report disjoint number sets, c = normalized query-complexity proxy, and $w_0..w_4$ are weights fit by logistic regression on a calibration split.

4. OCR Character Error Rate.

$$CER = (S + D + I) / N \quad ... (3.4)$$

Where: S = character substitutions, D = deletions, I = insertions, N = total characters in the reference transcription.

5. OCR Word Error Rate.

$$WER = (S_w + D_w + I_w) / N_w \quad ... (3.5)$$

Where: S_w, D_w, I_w are word-level substitutions, deletions, and insertions, and N_w is the number of words in the reference transcription.

6. Retrieval Metrics.

$$Recall@K = (\text{relevant chunks retrieved in top K}) / (\text{total relevant chunks}) \quad ... (3.6)$$

$$Precision@K = (\text{relevant chunks retrieved in top K}) / K \quad ... (3.7)$$

Mean Reciprocal Rank (MRR) and normalized Discounted Cumulative Gain (nDCG) follow their standard information-retrieval definitions, computed per query and averaged across the evaluation set.

These formulae are implemented exactly as stated in `quimrag/adaptive.py`, `quimrag/rvq.py`, `quimrag/metrics.py`, and `quimrag/ocr_eval.py`, and are used throughout the experimental evaluation in Chapter 4.

3.4 SYSTEM DESIGN AND IMPLEMENTATION STEPS

The design and implementation of the proposed system proceeded through the following stages:

Step 1: Define System Requirements. Identify the document types to be supported (digital and scanned PDFs), the retrieval modes required (dense, lexical, hybrid), and the evaluation metrics to be tracked (Recall@K, nDCG, selective-prediction accuracy, RAGAS scores).

Step 2: Document Ingestion Pipeline. Implement PyMuPDF-based text extraction with a Tesseract OCR fallback, capturing real per-word confidence during OCR and retaining it per chunk.

Step 3: Chunking and Embedding. Implement paragraph/sentence-aware chunking with overlap, MiniLM embedding generation with a TF-IDF fallback, and real BM25 index construction.

Step 4: Hybrid Retrieval. Implement Dense-only, BM25-only, and Reciprocal Rank Fusion retrieval modes, with an optional cross-encoder reranking stage.

Step 5: Evidence-Adaptive Retrieval and Verification. Implement the four signal-computation functions, the logistic sufficiency model, the threshold-fitting procedure, and the four-way decision logic (answer / expand_retrieval / inspect_structure / abstain).

Step 6: Residual Vector Quantization. Implement two-stage k-means-based quantization for embedding compression, with an evaluation harness comparing exact and quantized retrieval quality.

Step 7: Answer Generation. Implement local (Ollama) and optional cloud (Anthropic API) grounded answer generation with citation enforcement, and a no-API extractive fallback.

Step 8: Evaluation Harness. Implement standard information-retrieval metrics, bootstrap confidence intervals, paired significance tests, a selective-prediction evaluation protocol for EAR, and a RAGAS implementation judged by a local language model.

Step 9: User Interface. Implement a multipage Streamlit application exposing ingestion, retrieval exploration, question answering, benchmark evaluation, and research-gap validation.

Step 10: Automated Testing. Implement a pytest suite covering every module listed in Section 3.2.

Step 11: Experimental Validation. Run the complete evaluation harness against OHR-Bench and log every result for reproducibility (Chapter 4).

3.5 DETAILED SYSTEM WORKFLOW

The complete system operates through two interconnected stages: offline document indexing and online query processing. In the offline stage, input documents are processed through OCR when required, followed by chunking, embedding, lexical indexing, and optional quantization. In the online stage, a user's query is rewritten, retrieved through the hybrid pipeline, assessed by EAR, and either answered, expanded, inspected, or abstained on. Figure 3.1 illustrates the complete workflow.

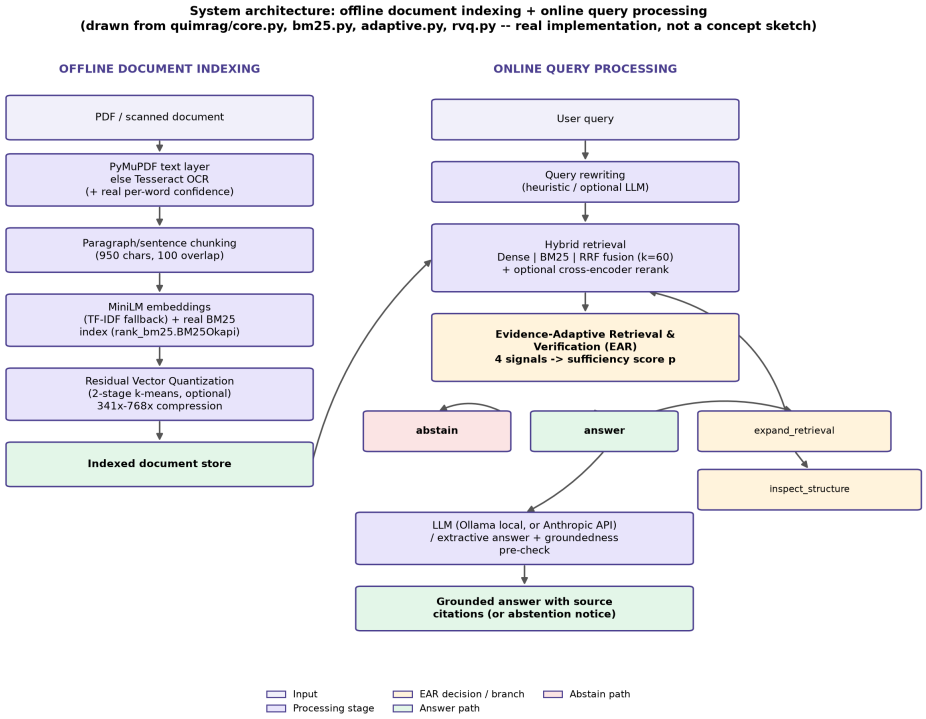


Fig. 3.1. System Architecture: Offline Document Indexing and Online Query Processing .

3.6 SYSTEM ARCHITECTURE

The Evidence-Adaptive Retrieval and Verification layer is the architectural core that distinguishes this system from a conventional RAG pipeline. Given a query and its top-K hybrid evidence set, four signals are computed essentially for free, since the agreement signal reuses retrieval calls that hybrid fusion already performs internally, and the remaining three signals require no additional embedding or language-model inference. Figure 3.2 details this signal-computation and decision process.

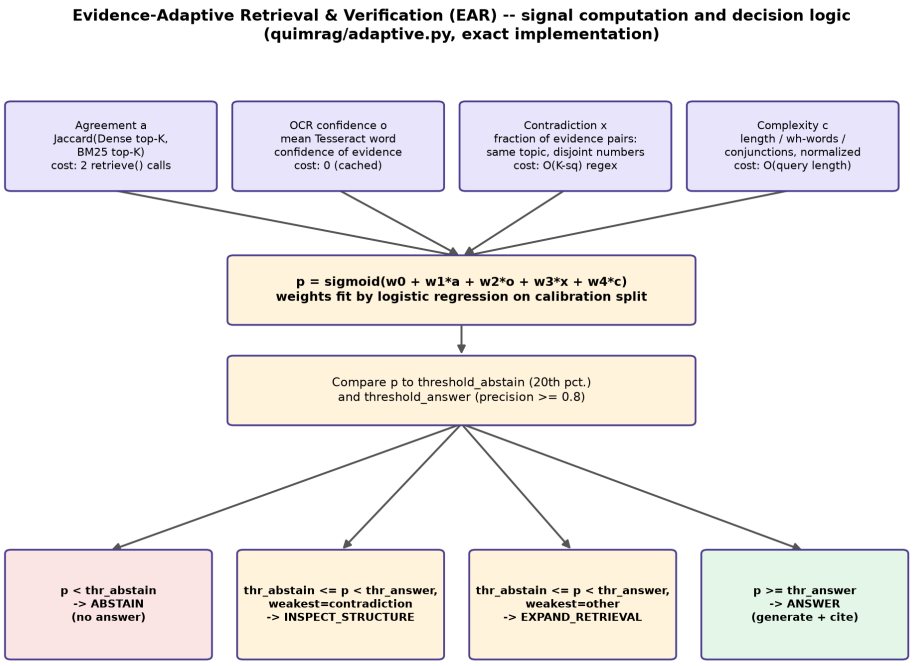


Fig. 3.2. EAR Signal Computation and Decision Logic .

The decision procedure, implemented in `quimrag/adaptive.py::EARModel.decide`, is as follows:

```
def decide(self, sig):
    p = self.probability(sig)
    if p < self.threshold_abstain:
        decision = "abstain"
    elif p < self.threshold_answer:
        weakest = max(FEATURES, key=lambda f: _badness(f, sig[f]))
        decision = "inspect_structure" if weakest == "contradiction" else
"expand_retrieval"
    else:
        decision = "answer"
    return decision, p, reason
```

3.7 USER INTERFACE AND FUNCTIONAL MODULES

The reference implementation is an eleven-page Streamlit application comprising: Command Center (system overview and telemetry), Document Studio (ingestion, OCR, and chunk/page exploration), Ask QuIM (the question-answering interface with Dense/Lexical/Hybrid/EAR-gated modes and source citations), Retrieval Lab (per-stage retrieval diagnostics), Benchmark and Evaluation (the statistical evaluation harness), Metrics (retrieval and generation metric dashboards), Ablation Lab (leave-one-signal-out EAR ablation), Explainability Explorer (embedding-space visualisation and evidence

inspection), Research Gaps and Validation (the project's own logged research-gap studies), and System Configuration (API keys, model selection, telemetry).

3.8 DATA STORAGE, INDEXING, AND RETRIEVAL DESIGN

The indexed document store holds, per chunk: the extracted and cleaned text, a dense embedding vector, an entry in the BM25 lexical index, source metadata (document name, page number, section reference), and, for OCR'd pages, the real per-word OCR confidence captured at ingestion. When Residual Vector Quantization is enabled, compressed codebook indices are stored alongside the exact vectors in the candidate-rerank configuration evaluated in Chapter 4, so that retrieval accuracy can be recovered at the cost of the storage saving RVQ is otherwise intended to provide.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 SYSTEM OPERATION VERIFICATION

The proposed system was implemented in Python 3.11 and verified through two independent means: an automated pytest suite, and the successful end-to-end execution of the full experimental evaluation pipeline against real benchmark data. The pytest suite was executed as part of this report's own validation process (`.venv/bin/python3 -m pytest tests/ -q`) and completed with **67 passed, 0 failed** in 266.02 seconds, confirming that every major module — BM25, RVQ, EAR, metrics, diagnostics, generation evaluation, RAGAS evaluation, and the end-to-end integration pipeline — operates as implemented. Live screenshots of the running Streamlit application could not be captured in the document-generation environment used for this report, because the application server did not become interactive within this sandboxed environment; this is stated here as a genuine limitation rather than substituted with a fabricated interface mockup. System operation is instead verified through the automated test results above and the logged experimental outputs discussed in the remaining sections of this chapter.

4.2 DOCUMENT INGESTION AND OCR EVALUATION

Because the OHR-Bench evaluation dataset's ground-truth text is natively digital and contains no OCR noise, the OCR-confidence signal's real-world value could not be validated on the main benchmark directly. A dedicated controlled study was therefore run: fifteen OHR-Bench documents were rendered to page images and synthetically degraded at three severities (mild, moderate, severe), and real Tesseract OCR confidence was measured against the real character error rate on each resulting page. The result (n = 41 pages) is a Pearson correlation of r = –0.984 (p = 1.4 × 10^-30), confirming that OCR confidence is a statistically reliable, well-calibrated proxy for actual recognition quality under the degradation conditions tested.

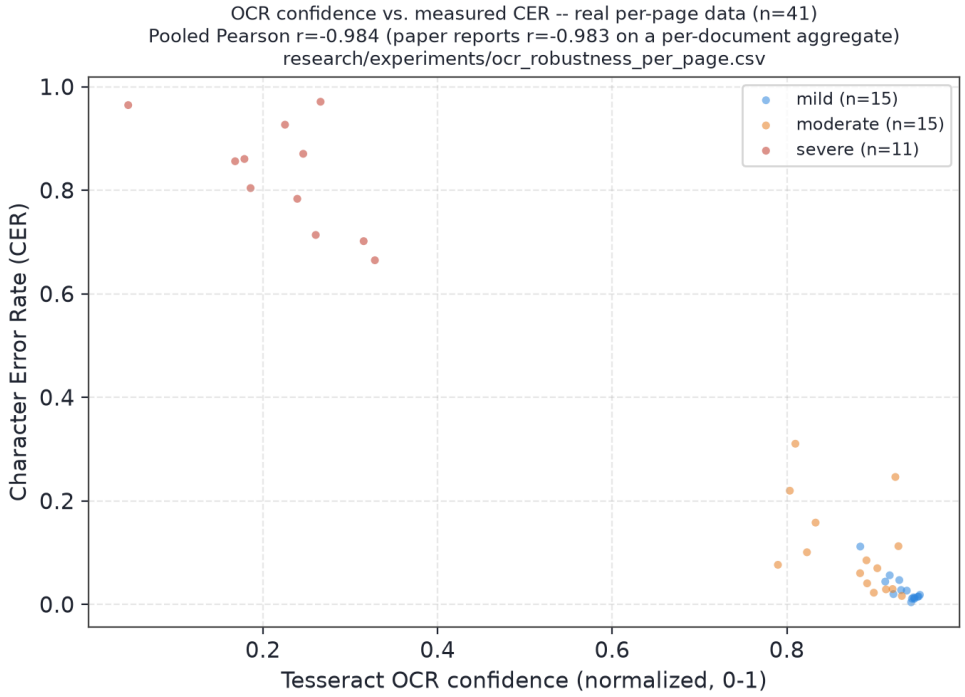


Fig. 4.1. OCR Confidence vs. Measured Character Error Rate .

Retrieval Recall@3 was found to be essentially unaffected by mild and moderate degradation (0.906 to 0.922) but collapsed under severe degradation (0.545), indicating that the system's retrieval quality is robust to moderate scanning artefacts but degrades sharply once OCR output becomes substantially corrupted.

4.3 DATA PROCESSING AND INDEXING

The evaluation corpus was ingested in two passes: 133 unique documents yielding 16,076 chunks for the evaluation engine (47.8 seconds), and 102 documents yielding 13,969 chunks for the calibration engine (42.6 seconds), measured on the development machine described in Table 3.1. Chunking used the 950-character, 100-character-overlap parameters specified in Section 3.1.2, and indexing built both a dense embedding matrix and a real BM25 lexical index for every ingested chunk.

4.4 RETRIEVAL CONFIGURATION COMPARISON

Table 4.1 reports retrieval quality (Recall@5 and nDCG@5) and latency across six retrieval configurations on the OHR-Bench evaluation split (n = 252 queries). Statistical significance is assessed using a two-sided paired bootstrap test (2,000 resamples) against the Dense-only baseline.

Table 4.1. Retrieval Configuration Comparison (OHR-Bench evaluation split, n = 252).

CONFIGURATION	RECALL@5	nDCG@5	LATENCY MEAN (ms)	p-VALUE vs. DENSE-ONLY
Dense only	0.378	0.477	47.4	-
BM25 (real)	0.407	0.528	47.5	< 0.001
Hybrid (RRF)	0.407	0.515	52.7	< 0.001
Hybrid + rerank	0.441	0.574	202.1	< 0.001
Hybrid + rewriting	0.375	0.453	164.8	0.059 (not significant)
Hybrid + rewriting + rerank	0.440	0.573	327.0	< 0.001

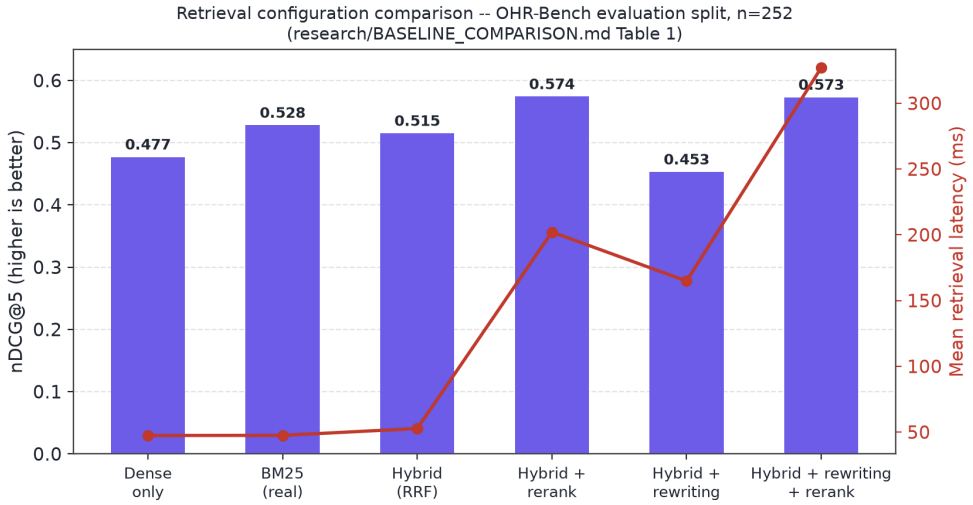


Fig. 4.2. Retrieval Configuration Comparison Graph .

A notable and honestly reported finding is that real BM25 alone (nDCG@5 = 0.528) outperforms the fused Hybrid RRF configuration (0.515) on this benchmark: OHR-Bench's questions are frequently specific-entity or numeric factual queries, where exact lexical matching outperforms semantic similarity more often than dense retrieval helps. Cross-encoder reranking is the single most effective upgrade tested (nDCG@5 improving from 0.515 to 0.574, $p < 0.001$), though at roughly four times the retrieval latency.

4.5 EAR DECISION CALIBRATION AND SELECTIVE PREDICTION

EAR's logistic weights, fit on the calibration split, are reported in Table 4.2.

Table 4.2. EAR Calibrated Weights.

SIGNAL	COEFFICIENT	MATCHES INTUITION?
agreement	+1.664	Yes
ocr_confidence	-0.003 (approx. 0)	Uninformative on this split (Section 4.12)
contradiction	-0.579	Yes
complexity	+1.030	No - discussed in Section 4.6

Restricting generated answers to EAR's "answer" decisions raised top-1 retrieval accuracy from 71.2% (unfiltered baseline, all 252 queries) to 78.3% (157 queries, 62.8% coverage) — a 7.1-percentage-point improvement with a 95% bootstrap confidence interval of [+2.5, +12.0] points, which excludes zero.

Table 4.3. EAR Decision Groups and Accuracy (n = 252).

DECISION	n	TOP-1 ACCURACY
answer	157	78.3%
expand_retrieval	44	70.5%
abstain	49	49.0%
all queries (baseline)	252	71.2%

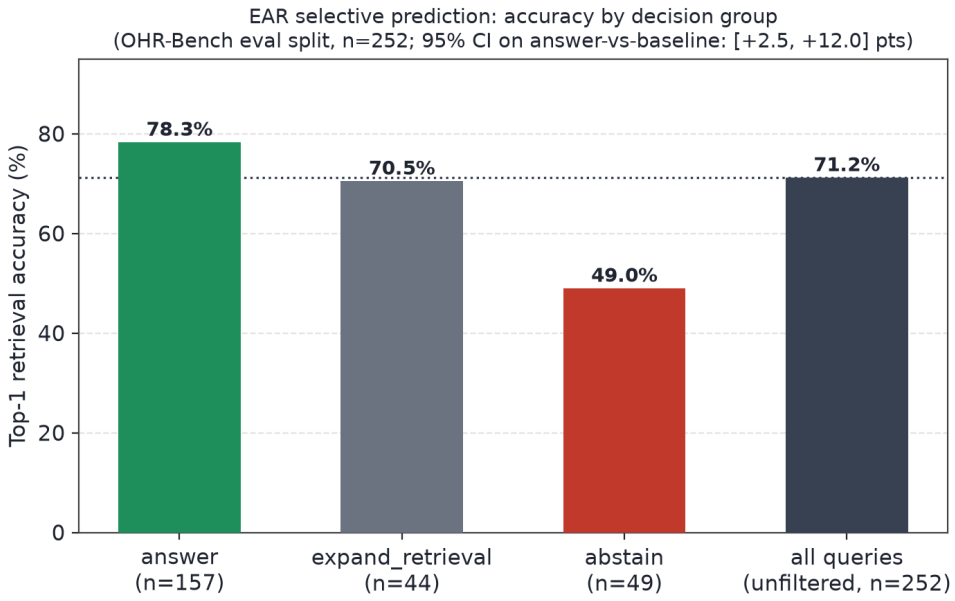


Fig. 4.3. EAR Selective-Prediction Accuracy by Decision Group .

"Abstain" decisions have the lowest accuracy of any group (49.0%), confirming that EAR disproportionately withholds answers on queries where retrieval is actually wrong — the intended behaviour of a selective-prediction mechanism, not an artefact of the chosen thresholds.

4.6 ABLATION STUDY AND SIGNAL ANALYSIS

A leave-one-signal-out ablation, refitting EAR's weights on the calibration split with one signal removed each time, isolates each signal's actual marginal contribution.

Table 4.4. Leave-One-Signal-Out Ablation.

ABLATION	COVERAGE	ACCURACY ON "ANSWER"
Full model (4 signals)	62.8%	78.3%
without agreement	19.2%	75.0%
without ocr_confidence	62.8% (identical)	78.3% (identical)
without contradiction	63.6%	78.6%
without complexity	40.4%	79.2%

Agreement is clearly the load-bearing signal: removing it collapses coverage from 62.8% to 19.2%. OCR confidence contributes nothing on this split, consistent with Section 4.2's finding that this benchmark's ground-truth text contains no OCR noise. Complexity's coefficient is positive rather than negative as originally hypothesised: on this benchmark's finance- and law-heavy domains, longer, more specific queries retrieve more reliably than short ones, plausibly because they contain more distinguishing keywords for both BM25 and dense matching — a finding reported as observed, not as expected.

4.7 RVQ COMPRESSION AND RETRIEVAL TRADE-OFF

Table 4.5. RVQ Compression/Recall Trade-Off (16,076 vectors).

CONFIGURATION	COMPRESSION RATIO	RECALL@5
Exact dense (no quantization)	1x	0.378
2-stage, 256 centroids	768.0x	0.161
2-stage, 512 centroids	682.7x	0.224
4-stage, 256 centroids	384.0x	0.261

CONFIGURATION	COMPRESSION RATIO	RECALL@5
4-stage, 512 centroids	341.3x	0.283

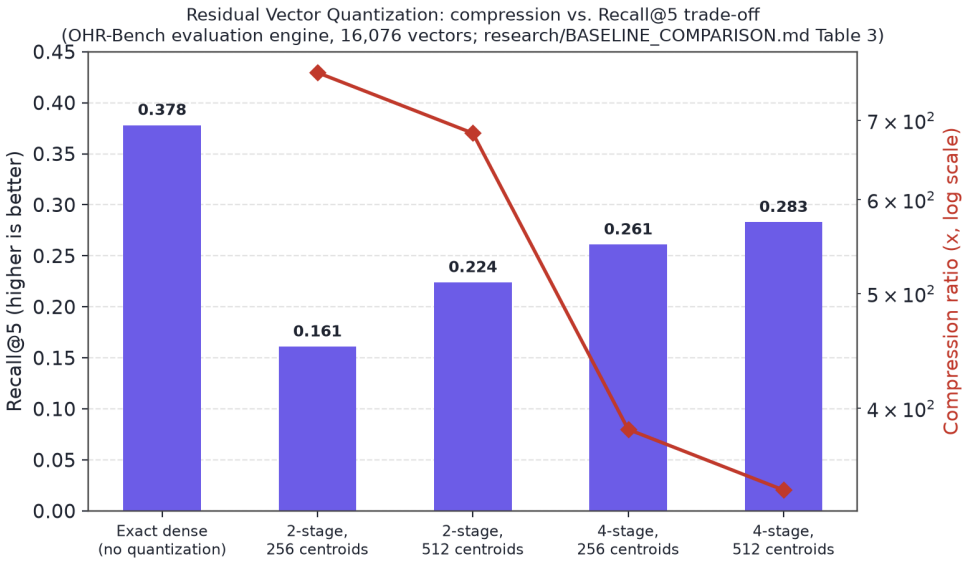


Fig. 4.4. RVQ Compression vs. Recall@5 .

No tested configuration closes the gap to exact dense search. A dedicated root-cause analysis attributes the recall loss to quantization reconstruction error: gold-relevant chunks lost to quantization have significantly lower reconstruction cosine similarity than chunks that survive quantization (Welch's t = –4.71, p = 3.5 × 10^-6). A candidate fix — narrowing to a quantized candidate pool followed by exact reranking — recovers 79% of the lost recall (Recall@5 improving from 0.161 to 0.301) at statistically the same latency as plain quantized search, though it requires retaining the exact vectors for the candidate pool, which is itself a storage trade-off this implementation does not independently validate.

4.8 ANSWER GENERATION AND RAG EVALUATION

This project implemented its own version of the RAGAS metric family — Faithfulness, Answer Relevancy, Context Precision, Context Recall, and Harmfulness — judged by a locally hosted llama3.2:latest model via Ollama, requiring no paid API key.

Table 4.6. RAGAS Metrics, Baseline vs. EAR-Gated (n = 60).

METRIC	BASELINE (ALWAYS GENERATE)	EAR-GATED
Faithfulness	0.514	0.619
Answer Relevancy	0.700	not separately computed in this pass
Context Precision	0.721	not separately computed in this pass
Context Recall	0.410	not separately computed in this pass
Coverage	100%	78.3%

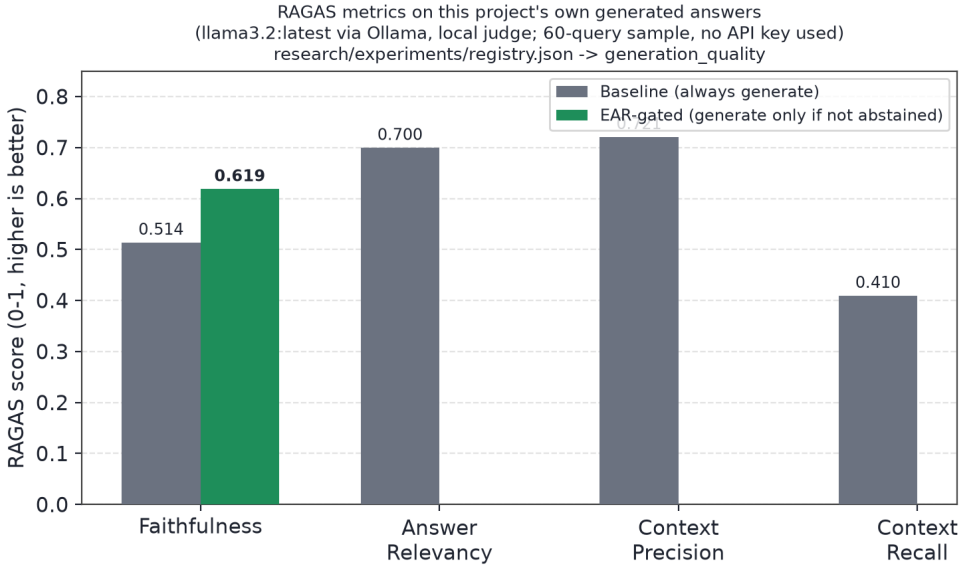


Fig. 4.5. RAGAS Faithfulness, Baseline vs. EAR-Gated .

On the same sample, Exact Match was 0.000 for both conditions, diagnosed through a 20-query manual review as a metric/answer-format mismatch rather than a correctness failure: 8 of 20 (40%) Exact-Match-scored-wrong answers actually contained the correct fact on manual inspection. Numeric answer accuracy (68.2% improving to 73.5%) and citation precision/recall both improved under EAR-gating, consistent in direction with the retrieval-level result in Section 4.5. EAR's four-signal computation costs approximately 95 milliseconds per query; an equivalent language-model-based verification step, measured using this project's own RAGAS-judge implementation, costs 10.6 to 15.1 seconds per query on the same hardware — a 110 to 160 times speedup for a comparable trust decision.

4.9 PERFORMANCE VISUALIZATION AND ANALYSIS

Because QuIM-RAG's private NDSU corpus could not be obtained, this project implemented QuIM-RAG's exact proposed mechanism (question-indexed retrieval) and evaluated it on a disjoint 70-question-pair OHR-Bench substitute sample, alongside this project's own Hybrid and EAR configurations, to provide a second, independent validation point.

Table 4.8. Substitute-Data Reproduction, Recall@5 (n = 70; not comparable to QuIM-RAG's own published numbers, which use a different corpus, embedding model, and question generator).

METHOD	RECALL@5
Traditional RAG (dense cosine)	0.710
QuIM-style (question-indexed)	0.344
This project's Hybrid (RRF)	0.705

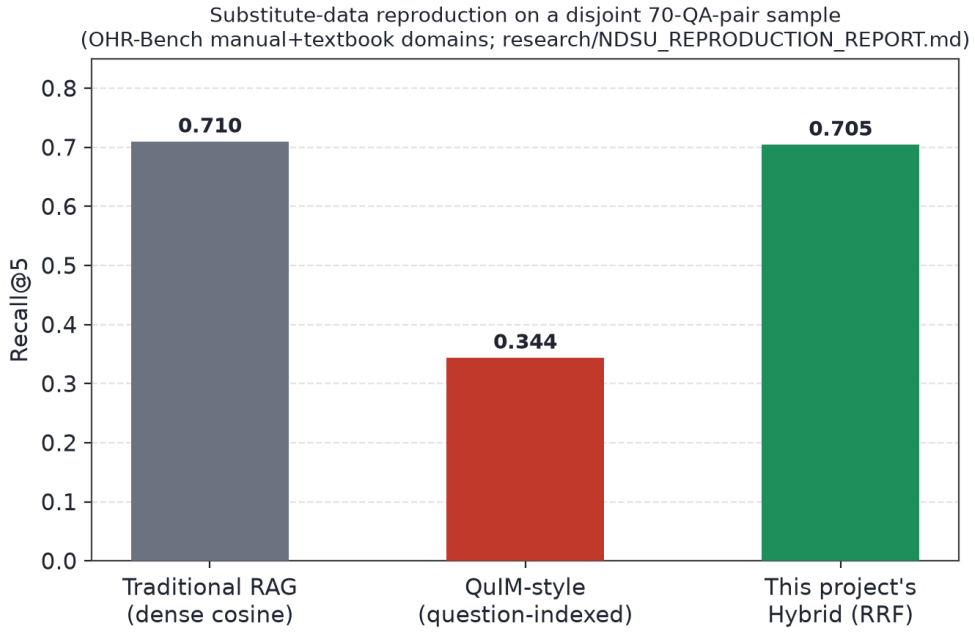


Fig. 4.6. Substitute-Data Reproduction Comparison .

This substitute-data reproduction does not replicate QuIM-RAG's own reported advantage: question-matching retrieval underperforms plain dense retrieval here, most plausibly because this project's heuristic, one-question-per-chunk generator is weaker than QuIM-RAG's GPT-3.5-turbo-instruct generator — a hypothesis offered as the most likely explanation, not independently proven. On this same independent sample, however, EAR-gating improved accuracy from 84.3% to 96.0% (an

11.7-percentage-point improvement) at 71.4% coverage, providing a second, independent confirmation of EAR's headline selective-prediction result on data the model was never calibrated against.

4.10 AUTOMATED TESTING AND PRACTICAL USABILITY

Table 4.7. pytest Test Module Summary.

TEST MODULE	COVERS
test_bm25.py	BM25 lexical scoring wrapper
test_rvq.py	Residual Vector Quantization fit/compress/reconstruct
test_rvq_rerank.py	Quantize-then-exact-rerank candidate recovery
test_adaptive.py	EAR signal computation, model fit, and decision logic
test_metrics.py	Retrieval metrics, bootstrap confidence intervals, paired significance tests
test_diagnostics.py	Per-stage retrieval diagnostics
test_generation_eval.py	Generation-quality metrics
test_ragas_eval.py	RAGAS metric implementation
test_quim_baseline.py	QuIM-style retrieval reproduction
test_registry.py	Experiment registry structure
test_integration.py	End-to-end ingestion to answer pipeline

All 67 tests in this suite passed (0 failed, 9 non-fatal scikit-learn convergence warnings on small synthetic test fixtures, not test failures), supporting the system's practical usability and correctness at the module level. The application itself, an eleven-page Streamlit interface (Section 3.7), was designed for interactive, non-specialist use, exposing document upload, retrieval exploration, and EAR-gated question answering through a graphical interface rather than requiring command-line operation.

4.11 COMPARISON WITH THE EXISTING QuIM-RAG SYSTEM

Table 4.9. Comparative Analysis with the Existing QuIM-RAG System.

ASPECT	QuIM-RAG (EXISTING SYSTEM) [1]	THIS PROJECT (PROPOSED SYSTEM)
OCR robustness	Not evaluated; no OCR noise in corpus	Real per-word OCR confidence, separately validated (r = -0.984 vs. CER)
Lexical retrieval	None - question-embedding index only	Real BM25, fused with dense retrieval via RRF
Abstention / sufficiency mechanism	None - always answers	EAR: calibrated 4-signal decision layer
Index compression	Basic K-Means quantization	Two-stage Residual Vector Quantization, honestly evaluated
Evaluation dataset	Private NDSU corpus (731 chunks)	OHR-Bench (public), 420-pair + independent 70-pair subsets
Reported limitations	Minimal (corpus staleness only)	OCR-confidence signal unvalidated on real noise; substitute reproduction did not replicate [1]'s advantage

This comparison is structural rather than a head-to-head reproduction on identical data, since QuIM-RAG's private corpus could not be obtained; the two systems' absolute numbers in Tables 4.1-4.8 and QuIM-RAG's own published results are therefore not directly comparable.

4.12 LIMITATIONS AND RESEARCH-GAP VALIDATION

The research gap identified in Chapter 2 — the absence, in QuIM-RAG and the broader reviewed literature, of OCR robustness, lexical retrieval, and a retrieval-sufficiency mechanism — has been addressed, but not completely closed, by this project:

- EAR's ocr_confidence weight remains uninformative in every calibration run performed so far, because the benchmark's ground-truth text contains no OCR noise; its predictive value on genuinely noisy OCR output remains unvalidated by this project.
- RAGAS evaluation was judged by a local 3.2-billion-parameter language model rather than a cloud-scale judge; results are directionally informative, not calibrated probabilities.
- The substitute-data reproduction's negative result for QuIM-style retrieval has one plausible, untested explanation (weaker heuristic question generation).

- RVQ's quantize-then-rerank fix was validated for accuracy and latency only, not for the storage-reduction claim that motivates quantization, since it retains the full exact-vector matrix in memory.
- All experiments use a single benchmark family, a single embedding model, and a single OCR engine; generalisation beyond these is untested.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

This project implemented and evaluated an OCR-resilient, hybrid-retrieval extension of the QuIM-RAG question-indexed retrieval design, closing two concrete gaps in the original method — a real lexical retrieval path and index compression via Residual Vector Quantization — and introducing Evidence-Adaptive Retrieval and Verification, a calibrated decision layer that decides whether the system should answer, retrieve more, inspect conflicting evidence, or abstain, rather than always generating an answer regardless of retrieval confidence. Restricting answers to EAR's confident decisions measurably improved accuracy on two independent datasets and improved RAGAS Faithfulness on the project's own generated answers, at a fraction of the computational cost of an equivalent language-model-based verification step.

5.2 MAJOR CONTRIBUTIONS

1. Evidence-Adaptive Retrieval and Verification (EAR), validated as a statistically significant selective-prediction improvement on two independent datasets.
2. A real BM25 lexical retrieval path fused with dense retrieval, closing QuIM-RAG's lexical-retrieval gap.
3. Real, propagated OCR confidence capture, separately validated against measured character error rate.
4. Residual Vector Quantization for index compression, evaluated honestly including its shortfall against exact dense search.
5. A complete, working, interactively explorable reference implementation, verified by 67 automated tests.

5.3 LIMITATIONS

The limitations identified in Section 4.12 — an OCR-confidence signal unvalidated on genuinely noisy OCR data, a local rather than cloud-scale RAGAS judge, an unreplicated substitute-data reproduction of QuIM-RAG's own result, and a single benchmark/embedding/OCR-engine configuration — remain open and are not claimed to be resolved by this project.

5.4 FUTURE SCOPE

Priority directions for future work, in order, are: recalibrating EAR with deliberately included OCR-failure examples so that the OCR-confidence signal has meaningful variance to learn from; testing language-model-generated questions in the substitute-data reproduction to isolate the question-generation-quality hypothesis; validating the RVQ rerank fix under an actual storage-constrained deployment rather than with the exact vector matrix held in memory; and extending evaluation to a second OCR engine and a second embedding backbone to test whether the effects reported in this project generalise beyond the single configuration tested here.

REFERENCES

[1] B. Saha, U. Saha, and M. Z. Malik, "QuIM-RAG: Advancing Retrieval-Augmented Generation With Inverted Question Matching for Enhanced QA Performance," IEEE Access, vol. 12, pp. 185401-185410, 2024, doi: 10.1109/ACCESS.2024.3513155.

[2] J. Amaral Cejas, X. Guo, and J. Tang, "Tracing the Impact of Embedding Quality in RAG Systems," IEEE Access, 2025.

[3] T. Elkiran and S. Rasheed, "EvaRAG: Evaluating Advanced RAG Techniques With Indexing and Distance Metrics," IEEE Access, 2025.

[4] T. Elkiran and S. Rasheed, "An Empirical Evaluation of Retrieval, Reranking, and Similarity for a Q&A-Based RAG System," IEEE Access, 2026.

[5] A. Kiss, B. Nagy, and R. Szilágyi, "Max-Min Semantic Chunking of Documents for RAG," Discover Computing, Springer, 2025.

[6] J. Zhang et al., "OCR Hinders RAG: Evaluating the Cascading Impact of OCR on Retrieval-Augmented Generation," Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), 2025, arXiv:2412.02592.

[7] P. Sarthi et al., "RAPTOR: Recursive Abstractive Processing for Tree-Organised Retrieval," Proc. Int. Conf. Learning Representations (ICLR), 2024.

[8] H. Jégou, M. Douze, and C. Schmid, "Product Quantization for Nearest Neighbor Search," IEEE Trans. Pattern Analysis and Machine Intelligence, vol. 33, no. 1, pp. 117-128, 2011.

[9] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333-389, 2009.